

NEB-X: Train Condition Monitoring

Problem Statement 3, all four subsystems

1 What we built

One web app that a depot engineer can use without touching code. Upload the recordings from a system, get an answer, read why, and download a maintenance handoff package. It covers all four subsystems, and it is the same app that produced every row of our `predictions.zip`.

Subsystem	Task	Metric	Our validation figure
Door	Segment cycles, classify each	IoU-weighted F1	1.000 accuracy, 110 cycles
ACV	Rank cars by leak likelihood	Linear rank-decay	faulty car first, 5 of 5 cases
Rail	Normal / Side I / Side II	Macro F1	0.750 ± 0.114 , 0.757 pooled
SHM	Cumulative fatigue damage	$\max(0, 1 - \text{MAPE})$	MAPE 2.52%, score 0.9748

Live at <https://neb-x-357243228666.asia-southeast1.run.app>; `./scripts/app.sh` runs it locally.

2 The app and its pipeline

Six steps, in the order a user meets them:

1. **Choose a system.** A board shows the four systems and whether each has a model loaded, so an unavailable one says so rather than failing on the first click.
2. **Add recordings.** Drag and drop; each system states the file types it reads. Filenames are preserved verbatim, because `file_id` in the submission is the source filename and a reformatted one matches nothing.
3. **Assess.** The batch runs in the background, one system at a time, so the user can switch to another system while the first finishes.
4. **Read the answer.** A verdict panel gives the headline, a severity colour and one line of detail. *Answer* is the plain-language view; *Details* steps through the model's own workings one panel at a time: per-cycle strips, ranked bars, threshold rows, time series.
5. **Keep the session.** Every system keeps its uploads, its latest assessment and its selected cycle while the user moves between systems.
6. **Hand off.** One button builds a ZIP: a plain-text summary, an offline printable HTML report with charts, per-system findings as CSV with evidence charts, and a `manifest.json` carrying assessment times, source filenames and checksums. No raw recordings, no model binaries, and systems with nothing to report are documented as such.

Every explanation the app draws is computed by the same functions that build the feature vector, so an explanation cannot drift from the prediction it explains.

3 How each subsystem is trained

Door. The controller only samples while a door moves, so cycles are separated by silence: rows are 20 ms apart inside a cycle and the recorder goes quiet for 10 s or more between them. Splitting on that gap reproduces all 110 labelled cycles exactly, which makes every IoU in the official metric 1.0 and collapses the score to plain accuracy. Classification then uses one feature: a cycle's mean motor current divided by the median of its own operation *within its own recording*. Absolute milliamps do not transfer between doors (info kit §1.2.2); a cycle's excess over its own door's typical cycle does. Scaling all currents by 0.85–1.20× costs a raw threshold up to 80 of 110 cycles and costs this feature nothing. The estimator is a gradient boosting classifier, effectively a stump that derives its own split, kept over a hardcoded threshold so the app has a probability to show. Validation is repeated stratified 5-fold, 10 repeats.

ACV. Nothing is fitted. Each car reports its cabin temperature and its cooling setpoint; subtract, average over the file, rank descending. A healthy car sits at or below its target, a leaking car runs above its own target because it cannot keep up. That single signal ranks the true faulty car first in all

five cases sharing the held-out file’s schema. Every car is always ranked, never dropped: the organisers score an omitted car as zero.

Rail Corrugation. One file is one second of an 8-car train crossing a section of track, 64 axle boxes per rail at 10 kHz. Both rails are crossed by the same train at the same speed in the same second, so the *contrast between the two sides* cancels the confounders the info kit names (speed, ballast noise, track elasticity) and what remains is genuinely asymmetric. Energy is binned by ripple *wavelength* rather than frequency, because a wheel crossing corrugation rings at speed/wavelength and speed varies tenfold across these files; that choice alone is worth +0.06 macro F1. Each side’s 32 boxes are aggregated by max and p90, never median, because only some boxes cross the corrugated stretch and a median is built to discard exactly those. Estimator: class-balanced gradient boosting, 228 features, repeated stratified 5-fold with 10 repeats. A regularised linear baseline ties it, and five hyperparameter variations fail to beat it, so the simple model is the one we ship.

SHM. Physics, with two fitted numbers. Rainflow counting reduces each stress history to closed cycles, and Miner’s rule sums them into damage $D = S(m)/C$ with $S(m) = \sum_i n_i \sigma_i^m$. The exponent m and coefficient C are chosen to minimise MAPE directly, the metric this subsystem is scored on, rather than least squares, which would chase the large-damage files and leave the small ones wrong. The search is coarse then fine because the optimum in m is sharp. Validation is leave-one-out over the 64 labelled files, honest here only because the model has two parameters; with a high-capacity model it would leak.

4 How it comes together

Dependencies point one way: `common` $\leftarrow$ `subsystem` $\leftarrow$ `app`. No subsystem imports another, shared logic moves down into `common/` and never sideways, and every path in the project is owned by one file (`src/common/config.py`) so the trainer and the predictor cannot disagree about where a checkpoint lives.

The app never branches on which subsystem is selected. It asks for a capability and gets one or does not: `services.py` resolves each subsystem’s `predict` (required) and `explain` (optional) by `importlib`, over the same input paths. Explanations come back as plain dictionaries tagged with a `kind`, and the renderer draws whatever arrives. A fifth subsystem would be added without editing a line of the app.

The submission is one command. `./submit.sh` runs every held-out test input through the same `predict` the app calls, validates each CSV against the shipped schema, and refuses to package one that does not match, because the organisers score the zip as-is. It then assembles the team folder: `predictions.zip` flat at the top, the runnable app with its checkpoints, the demo video, and `Optional_Items/` holding this write-up and each subsystem’s code and model. `./scripts/deploy.sh` serves that same app on Cloud Run.

5 Assumptions and limits

- Splits are by file, never by window, and every validation figure quoted above is out-of-fold. Nothing is tuned against the held-out test inputs.
- Rail: no fault file in training is slower than 9.70 m/s while 133 of 234 `Normal` files are, so “slow implies Normal” is available for free. On the 138 files fast enough to remove that shortcut the score is **0.770**, above the headline rather than below it. We quote both figures rather than the flattering one.
- Rail: on this data, 36 columns of shuffled noise buy +0.012 macro F1, so any added feature block must beat its own shuffled control before it ships. Side I, 14 files, stays the weakest class at 0.515.
- ACV is a rule, not a fitted model, validated on five cases. Cheap to audit, but untested against a fleet carrying more than one fault.
- SHM damage values describe the damage contributed by individual recordings, not remaining asset life. The app’s outputs are decision support for maintenance review, not a fitness-for-service decision.